

Velvet Vista

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project: Enterprise Luxury Watch E-Commerce Showcase	Architecture: Strict Object-Oriented Programming (OOP)
Frameworks: Angular (Frontend) & NestJS (Backend)	Infrastructure: Cloudflare Distributed Edge & Supabase
Document Version: 1.0.0	Date: May 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes a comprehensive architectural blueprint, system requirements, and structural specifications for the **Velvet Vista** e-commerce ecosystem. Velvet Vista is an enterprise-grade digital showcase platform engineered to demonstrate full-stack software development excellence. The system enforces absolute compliance with Object-Oriented Programming (OOP) design patterns, automated DevOps CI/CD principles, robust accessibility standards, and zero-cold-start edge cloud scaling.

1.2 Scope

Velvet Vista functions as a ultra-luxury virtual showroom and transaction simulation network for high-end horology timepieces. The application architecture is cleanly segmented into two distinct decoupled computing domains:

- **Velvet Vista Client (Frontend):** A high-performance Angular client application hosted and compiled via Cloudflare Pages. It utilizes structured, class-based reactive architectures and strict data encapsulation models.
- **Velvet Vista Core API (Backend):** An enterprise NestJS application micro-engine deployed inside a serverless Cloudflare Workers runtime utilizing modern

nodejs_compat

capability matrices, routing workflows securely to a multi-region Supabase PostgreSQL instance.

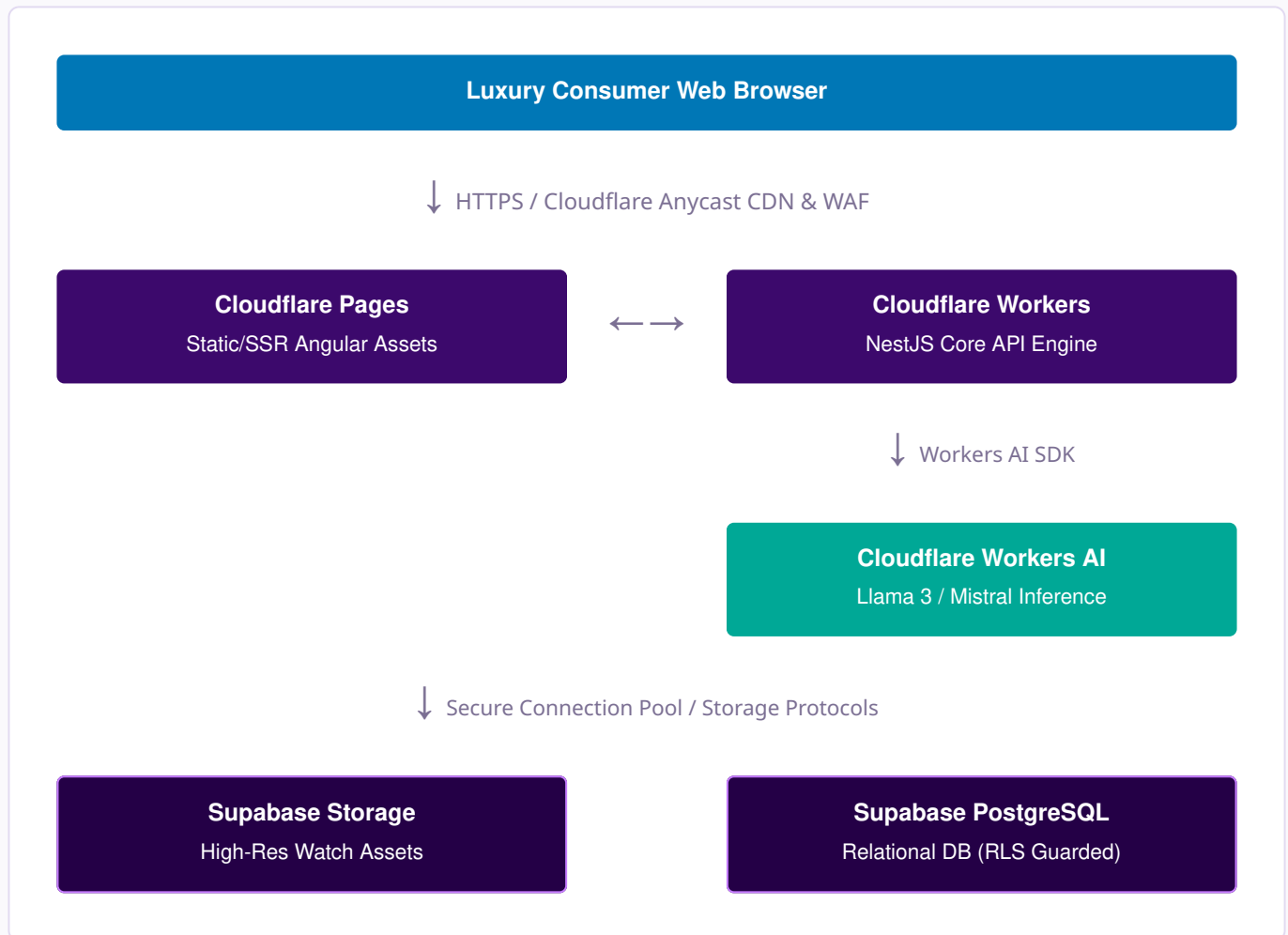
1.3 Intended Audience

This technical manual is compiled for software architects, full-stack engineers, QA automation professionals, and AI development co-pilots tasked with generating code or auditing compliance within the Velvet Vista ecosystem repository boundaries.

2. System Architecture & Infrastructure Design

The operational infrastructure of Velvet Vista is mathematically and physically structured to eliminate cold-start latencies, provide worldwide multi-region availability, and operate fully within cost-efficient cloud free tiers. The entire infrastructure stack completely avoids regional geo-blocking restrictions, ensuring complete accessibility for development teams globally (including Syria under General License 25 frameworks).

2.1 Component Interaction Topology



2.2 Layer Responsibilities

- **Client Presentation Layer (Cloudflare Pages):** Responsible for static layout assembly, user interface response compilation, local accessibility enforcement, and instant client-side route tracking.
- **Edge Compute Layer (Cloudflare Workers):** Executes business logic micro-services within a lightweight V8 isolate container. Hosts the NestJS framework shell, validating request schemas and generating deterministic JSON data payloads.
- **Persistent Relational Layer (Supabase PostgreSQL):** Enforces relational data completeness with transactional safety. Row-Level Security (RLS) filters read and write attempts against validated JSON Web Token (JWT) cryptographic signatures.

3. Detailed Functional Specification (MoSCoW Framework)

PRIORITY LEVEL	REQUIREMENT SPECIFICATIONS & ARCHITECTURE MANDATES
MUST HAVE Core Architecture	Strict Object-Oriented Programming (OOP): Codebases must exclude structural procedural scripts or un-encapsulated functions. All domain processes belong entirely to explicit classes or strict TypeScript interfaces. Inversion of Control & DI: Framework-level dependency injection handles lifecycle behaviors. Concrete services must resolve cleanly via explicit abstract interfaces or token definitions. Relational RLS Layer: Supabase PostgreSQL models strict schemas across Users, Products (Watches), Orders, and active Cart states, secured implicitly by Row-Level Security. Luxury Dual Themes: Dynamic style engine changing UI presentation tokens seamlessly: • <i>Dark Mode:</i> Obsidian Canvas (#0B0C10), Velvet Base (#4B0082), Aqua Accent (#00FFFF), Indigo Elements (#2A2B4D). • <i>Light Mode:</i> Paper Canvas (#FAFAFA), Slate Text (#1C1C1C)

PRIORITY LEVEL	REQUIREMENT SPECIFICATIONS & ARCHITECTURE MANDATES
	), Emerald Accent (#00875A). DevOps Automated Workflows: GitHub Actions automated runs on every Pull Request to main branch ensuring strict lint validation, Prettier syntax checks, and complete Jest unit execution blocks.
SHOULD HAVE High-Priority	Fuzzy Logic Search Service: Premium catalog discoverability must resolve fuzzy text typos efficiently without continuous database compute queries. It leverages an in-memory, edge-cached matrix calculation service using the Levenshtein Distance algorithm. Edge AI Chat Assistant: Integrated luxury virtual concierge context drawer streaming real-time tokens natively from Cloudflare Workers AI executing edge LLM model inference with zero external token billing hooks. Keyboard Trapping & Directives: Strict focus management and physical arrow key navigation overrides (ArrowUp , ArrowDown) for accessible modal views and sliding layout overlays.
COULD HAVE Advanced Standouts	Model Context Protocol (MCP) Node: Realization of an internal McpController providing JSON-RPC 2.0 definitions over Server-Sent Events (SSE). This permits external AI orchestrators to explicitly inventory the watch collection

PRIORITY LEVEL	REQUIREMENT SPECIFICATIONS & ARCHITECTURE MANDATES
	metadata smoothly. Predictive Static Prefetching: Interaction analytics measuring mouse hover intervals over luxury watch catalog items; triggers asynchronous asset compilation prior to click actions to drop UI load visual gaps to absolute zero.
WON'T HAVE Deferred Scope	Live Financial Payment Integration: Excludes active card verification endpoints or local financial networks. Uses a simulated deterministic mock state-machine tracking ledger for checking out operations. Compiled Desktop/Mobile Binaries: Focuses exclusively on highly optimized progressive web standards over dedicated platform native operating binaries.

4. Mathematical Modeling & Code Snippets

4.1 Fuzzy Logic Core Metric Formulation

The native

FuzzySearchService

assesses the text variation difference between user entry strings and database catalog parameters. For any given user query string q and watch description attribute a , the matching value model $Sim(q, a)$ is calculated using the following algorithmic ratio:

$$Sim(q, a) = 1 - \frac{Lev(q, a)}{\max(|q|, |a|)}$$

Where $Lev(q, a)$ defines the absolute Levenshtein Distance matrix calculation tracking the lowest count of single-character corrections (additions, updates, omissions) necessary to transition string q directly into string a . Items return in user lookups only if $Sim(q, a) \geq 0.65$.

4.2 Strict OOP Coding Layout (NestJS Edge Implementation)

Below is the standardized, production-ready class structural blueprint mapping our strict architectural requirements, property segregation rules, and method documentation headers:

```
/**
 * @class FuzzySearchService
 * @description Provides mathematical fuzzy matching capabilities for high-end catalog
 search.
 * @property {number} minimumMatchThreshold The baseline score required to return a
 product match.
 */
export class FuzzySearchService {
  // 1. PUBLIC PROPERTIES / CONFIGURATION SIGNALS
  public readonly engineVersionSignature: string = "v1.0.0";

  // 2. PRIVATE OPERATIONAL STATE
  private readonly minimumMatchThreshold: number = 0.65;

  /**
   * @method calculateLevenshteinDistance
   * @param {string} source The user input query string.
   * @param {string} target The product database attribute string.
   * @returns {number} The absolute edit distance between strings.
   */
  private calculateLevenshteinDistance(source: string, target: string): number {
```

```

const matrix: number[][] = [];

for (let i = 0; i <= source.length; i++) matrix[i] = [i];
for (let j = 0; j <= target.length; j++) matrix[0][j] = j;

for (let i = 1; i <= source.length; i++) {
  for (let j = 1; j <= target.length; j++) {
    if (source[i - 1] === target[j - 1]) {
      matrix[i][j] = matrix[i - 1][j - 1];
    } else {
      matrix[i][j] = Math.min(
        matrix[i - 1][j] + 1,    // Deletion
        matrix[i][j - 1] + 1,    // Insertion
        matrix[i - 1][j - 1] + 1 // Substitution
      );
    }
  }
}
return matrix[source.length][target.length];
}

/**
 * @method evaluateProductMatch
 * @param {string} query Raw user query string.
 * @param {string} attribute Target attribute (e.g., brand name, model series).
 * @returns {boolean} True if similarity index satisfies the minimum match
requirement.
 */
public evaluateProductMatch(query: string, attribute: string): boolean {
  const distance = this.calculateLevenshteinDistance(query.toLowerCase(),
attribute.toLowerCase());
  const maxLen = Math.max(query.length, attribute.length);
  if (maxLen === 0) return true;

  const similarityScore = 1 - (distance / maxLen);
  return similarityScore >= this.minimumMatchThreshold;
}
}

```

5. Automated DevOps Pipeline Mapping

The code verification workflow guarantees that zero malformed structural classes reach the master branch line. The workflow layout utilizes fully isolated validation containers executed automatically via GitHub runner architectures:


```
name: Velvet Vista Enterprise Validation Engine
on:
  pull_request:
    branches: [ main ]

jobs:
  verify-ecosystem:
    runs-on: ubuntu-latest
    steps:
      - name: Fetch Active Repository Branch State
        uses: actions/checkout@v4

      - name: Establish Modern Node Runtime Context
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Hydrate Runtime Dependencies
        run: npm ci

      - name: Audit Quality Guidelines & ESLint Mandates
        run: npm run lint

      - name: Validate Formatting Integrity via Prettier
        run: npm run format:check

      - name: Execute Isolated Unit Context Testing via Jest
        run: npm run test:unit
```